TP N° 8 Machine Learning & Text Mining
KNN

Considérons les programmes suivants :

```python
X = [[0], [1], [2], [3]]
y1 = [0, 0, 1, 1]
from sklearn.neighbors import KNeighborsClassifier
#valerur de k pour knn et choix de la distance
neigh = KNeighborsClassifier(n_neighbors=3,metric="euclidean")
neigh.fit(X, y1)
print('Knn predefinie-classification:',neigh.predict([[2.1]]))
from sklearn.neighbors import KNeighborsRegressor
knn_regression =KNeighborsRegressor(3)
y2 = [15.3, 8.6, 6.5, 4.29]
knn_regression.fit(X,y2)
print('Knn predefinie-regression:',knn_regression.predict([[2.1]]))
```

```python
from collections import Counter
import math
def mean(labels):
    """Calcul de la moyenne des k étiquettes"""
    return sum(labels) / len(labels)
def mode(labels):
    """Calcul du mode des k étiquettes"""
    return Counter(labels).most_common(1)[0][0]
def euclidean_distance(point1, point2):
    """Calcul de la distance euclidienne"""
    sum_squared_distance = 0
    for i in range(len(point1)):
        sum_squared_distance += math.pow(point1[i]-point2[i], 2)
    return math.sqrt(sum_squared_distance)
```

```python
def knn(data, query, k, distance_utilisee, choix_knn_regress_ou_classif):
    """Algorithme des k plus proches voisins kNN"""
    distances_et_positions = []
    for index, example in enumerate(data):
        distance = distance_utilisee(example[:-1], query)
        distances_et_positions.append((distance, index))
    #ici une liste de tuples sera triée selon les distances
    sorted_distances_et_positions =sorted(distances_et_positions)
    k_proches_distances_and_indices =sorted_distances_et_positions[:k]
    k_proches_classes =[]
    for dist_indice in k_proches_distances_and_indices:
        num=dist_indice[1]; k_proches_classes+=[data[num][1]]
    # 7. Si régression (choix_knn_regress_ou_classif = mean),
    #retourner la moyenne des k étiquettes
    # 8. Si classification, retourner le mode des k étiquettes
    return k_proches_distances_and_indices ,\
choix_knn_regress_ou_classif(k_proches_classes),k_proches_classes
```

```python
# Données-régression Colonne 0:taille (en cm)Colonne 1: poids (en kg)
from numpy import *
taille_poids =array([[ 167.66,  112.99],[ 182.38,  136.49],
       [ 176.97,  153.03],[ 173.96,  142.34],[ 172.86,  144.3 ],
       [ 175.19,  123.3 ],[ 177.99,  141.49],[ 178.53,  136.46],
       [ 173.15,  112.37],[ 169.55,  127.45]])
taille_en_cm=[161]
k=3; distance_utilisee=euclidean_distance; choix_knn=mean
R= knn(taille_poids, taille_en_cm, k, distance_utilisee, choix_knn)
knn_dist_et_indices, prediction_de_poids, knn_labels=R
print('Knn implementé pour regression:')
print(knn_dist_et_indices,'\n',prediction_de_poids,'\n',knn_labels)
from sklearn import neighbors
knn_regression = neighbors.KNeighborsRegressor(k)
X=[[e] for e in taille_poids[:,0]]; Y=[e for e in taille_poids[:,1]]
knn_regression.fit(X,Y)
print('Knn predefinie-regression:\n',knn_regression.predict([taille_en_cm]))
```

```python
# Données de Classification, Colonne 0: age  Colonne 1: aime jeux?
age_aimer_jeux_electroniques = array([[22,1],[23,1],[21,1],[18,1],[19,1],
                                      [25,0],[27,0],[29,0],[31,0],[45, 0]])
#age 33 ans aime-t-elle les jeux électroniques
age = [33]; k=5; distance_utilisee=euclidean_distance; choix_knn=mode
R= knn(age_aimer_jeux_electroniques, age, k, distance_utilisee, choix_knn)
knn_dist_et_indices, prediction_de_aimer_jeux, knn_labels=R
print('résultat knn implémenté pour la classification:')
print(knn_dist_et_indices,'\n',prediction_de_aimer_jeux,'\n', knn_labels)
from sklearn.neighbors import KNeighborsClassifier
neigh = KNeighborsClassifier(n_neighbors=k,metric="euclidean")
X=[[e] for e in age_aimer_jeux_electroniques[:,0]];
Y=[e for e in age_aimer_jeux_electroniques[:,1]]
neigh.fit(X,Y)
print('résultat knn prédéfinie pour la classification:')
print(neigh.predict([age]))
```

```python
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV,train_test_split
data=pandas.read_csv("d:\\images_chiffres_codees_niveau_de_gris.csv")
data=np.array(data); x=data[0:1000,1:]; y=data[0:1000,0];
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,
                                               random_state=0)
model=KNeighborsClassifier(n_neighbors=5,metric='manhattan')
model.fit(x_train,y_train); score=model.score(x_test, y_test)
print("score sur test-set par k=5 --> "+str(score*100)+'%')
distances=['euclidean','manhattan'] ; valeurs_de_k=np.arange(1,8)
parametres_grid={"n_neighbors":valeurs_de_k, 'metric':distances}
model=KNeighborsClassifier()
grid=GridSearchCV(model,parametres_grid,cv=3,scoring='accuracy')
grid.fit(x_train,y_train); print("best parameters:",grid.best_params_)
model=grid.best_estimator_  ; model.fit(x_train,y_train)
score=model.score(x_test, y_test)
print("score sur test-set-paramètres automatiques="+str(score*100)+'%')
```

Travail à faire :

1. En analysant et exécutant le premier programme, quelle est la différence entre KNN régression et KNN classification ?
2. Dans le deuxième programme, quel est l'intérêt des fonctions mode, mean ?
3. Dans les programmes 3 et 4, changer le nombre d'éléments le plus proche et la distance ? et noter les résultats de performances obtenus ?